Anomaly Detection Algorithm for Enhanced Fraud Identification in the Insurance Sector

Mehrab Atighi[a], Sayed Jamal Mirkamali[b], Amir T. Payandeh Najafabadi[a,1]

a Department of Actuarial Science, Faculty of Mathematical Sciences, Shahid Beheshti University, Evin, 1983969411, Tehran, Iran.

b Department of Mathematics, Faculty of Science, Arak University, 3848177584, Arak, Iran.

September 19, 2026

Abstract

Insurance-fraud screening requires effective detection under class imbalance and limited verification resources. Sequential Anomaly Detection (SAD) uses an anomaly ensemble to select claims for review, then trains eight supervised learners on verified records and updates them across successive batches. Three architectures differ in their initial screening: Simple-A uses nine anomaly detectors, Simple-B adds an entropy vote, and Multilayer applies entropy after anomaly consensus. We evaluate these architectures using ten repeated stratified random splits of Carclaims (15,420 records; 5.99% fraud) and Insurance Claims (1,000 records; 24.70% fraud). Recorded outcomes simulate verification after case selection, including random audits of non-alerted claims. At fixed voting thresholds $(3, 4)$, Simple-B achieves mean fraud F1 0.560 and average precision 0.491 on the second benchmark. Multilayer achieves F1 0.497 at a review fraction of 0.365. A frozen-model ablation, ten classifiers trained under a common acquisition policy, and a stricter threshold setting examine the effects on detection and workload. The findings support assessment of both outcomes when selecting a review policy; they do not establish universal architectural superiority or prospective generalizability to an independent insurer.

Keywords: insurance fraud; sequential updating; anomaly detection; entropy; verification workload; class imbalance.
MSC2010: 62D05, 62F10, 62N01, 65C05.

[1]amirtpayandeh@sbu.ac.ir

1 Introduction

Insurance fraud creates financial losses and places demands on claims investigation. Insurers must identify suspicious claims while allocating limited review resources [Hilal et al., 2022; Bolton and Hand, 2002]. This task is difficult because legitimate claims are heterogeneous and unusual observations are not necessarily fraudulent.

Machine-learning approaches address this problem through supervised classification, unsupervised screening, or combinations of the two [Ngai et al., 2011; West and Bhattacharya, 2016; Schrijver et al., 2024]. Their practical value depends on both detection performance and the number of claims requiring verification. Selective investigation also determines which labels become available for subsequent training.

Sequential Anomaly Detection (SAD) connects initial anomaly screening to verification and repeated supervised learning. The framework is intended to support a choice between detection performance and review workload. We examine this choice through the simple and multilayered SAD structures.

The contribution lies in the organization and evaluation of established components within a verification-driven workflow. Its *sequential* mechanism incorporates verified records from one batch into training before the ensemble screens the next batch. Its *multilayer* mechanism applies an entropy filter to the candidates selected by anomaly-detector consensus. Updating changes the information available for later prediction; layering changes which initial cases reach verification. Simple-A, Simple-B and Multilayer retain the original architectural alternatives: entropy is absent, contributes a flat vote, or forms a second screening layer, respectively.

The evaluation compares these three architectures, introduces a frozen-model control for sequential updating, and applies the same prescribed protocol to two benchmark datasets. Additional classifiers include an imbalance-aware forest and a recent fraud-motivated boosting approach. The experiments explicitly account for review workload and designate fraud as the positive class. Recorded outcomes simulate expert feedback and are revealed to the learner only after review selection. SAD uses fixed acquisition rules; the comparison with reinforcement learning therefore concerns workflow design rather than a learned reward-maximizing policy.

Section 2 positions SAD within the fraud-detection literature. Section 3 defines its components and update rules; Section 4 reports the benchmark experiments. Section 5 discusses the findings and their limitations.

2 LITERATURE REVIEW

The literature is grouped below by the information used for learning and the role of verification in each approach. Early statistical and data-mining reviews distinguish label-based detection from anomaly screening [Bolton and Hand, 2002; Ngai et al., 2011]; broader financial-fraud reviews describe their continuing complementarity [West and Bhattacharya, 2016; Hilal et al., 2022]. The systematic review of Schrijver et al. [2024] describes the prominence of supervised learning and oversampling in automobile-insurance fraud studies, as well as limitations in public data availability.

2.1 Supervised approaches

Supervised models use previously investigated claims to estimate a decision boundary or ranking. Regression, support-vector machines, forests, boosting, and neural networks are common choices [Moon et al., 2019; Hilal et al., 2022]. Insurance-specific work includes optimized boosting and interpretable assessment [Ding et al., 2025], practical model-selection guidance [Banulescu-Radu and Yankol-Schalck, 2024], and imbalance and missing-value handling [Khalil et al., 2024]. These methods can exploit claim attributes effectively when their training labels are sufficiently representative. Direct comparisons of supervised and unsupervised insurance models are also reported by Debener et al. [2023]. Selective historical investigation, however, can bias which examples are labeled.

In *Annals of Data Science*, Kamalov et al. [2023] examine feature selection under imbalance and propose a minority-sensitive criterion. Their work motivates explicit attention to fraud-class metrics here; we do not equate our unsupervised Gini screen with their supervised selection algorithm.

2.2 Unsupervised approaches

Anomaly methods rank observations according to deviations from a fitted notion of normality. Auto-insurance applications include spectral ranking [Nian et al., 2016] and unsupervised deep learning [Gomes et al., 2021]. Such approaches can propose cases before reliable labels exist, but unusual observations need not be fraudulent. Clustering methods also depend on representation and dissimilarity choices, as reviewed by Xu and Tian [2015] in *Annals*

of Data Science. This supports using heterogeneous signals while retaining verification of their interpretation.

2.3 Reinforcement-learning-based approaches

When verification is costly, a learned policy can decide which observations justify acquiring an outcome. In the framework studied by Nijhuis and van Lelyveld [2023], reinforcement learning guides this information-acquisition decision for outlier detection. Actions are evaluated through a reward signal. SAD instead uses fixed voting thresholds and a random audit fraction; it has no learned reward-maximizing query policy. Its sequential component is retraining after each reviewed batch, which can be tested against a frozen model.

2.4 Hybrid architectures

Hybrid approaches combine representations or learners, while graph methods use relationships among claims, people, or entities [Vorobyev, 2024]. PSO-XGBoost is a recent insurance-specific example of optimization combined with boosting [Ding et al., 2025]. Hassani et al. [2020], in *Annals of Data Science*, review deep-learning applications in banking, providing broader financial-domain context. That review is contextual evidence, not a direct insurance-fraud performance comparator.

2.5 Research gap and comparator scope

SAD addresses the connection between candidate generation, review, and iterative learning. It does not replace the need for representative labels or resolve all challenges in recent deep or graph-based systems. Table 1 distinguishes these roles. The numerical experiments include an explicitly adapted PSO-XGBoost comparator and balanced random forests in addition to eight component classifiers. We do not attribute our adapted implementation's numbers to the original publication.

Table 1: Comparative positioning by learning and acquisition mechanism.

Family	Representative source	Strength	Boundary relevant to SAD
Supervised	Ding et al.; Banulescu-Radu and Yankol-Schalck	Direct label-based discrimination	Needs representative verified labels
Unsupervised	Nian et al.; Gomes et al.	Initial screening without outcomes	Anomaly is not confirmed fraud
Reinforcement learning	Nijhuis and van Lelyveld	Values costly information	Requires a query/reward design
Hybrid/graph	Vorobyev; Ding et al.	Relational or optimized predictive signals	May require links or additional tuning
SAD	This study	Explicit review and batch update workflow	Retrospective simulation; policy-dependent labels

Table 2: Mechanisms of recent counterparts compared with SAD. Design comparison does not imply numerical reproduction.

Method	Main mechanism	Relation to sequential review	Relation to multilayer SAD
PSO-XGBoost [Ding et al., 2025]	Optimized boosted trees	Search tunes parameters; it does not implement batch review	No post-consensus entropy gate specified
Unsupervised deep learning [Gomes et al., 2021]	Learned anomaly representation	Does not by itself define SAD's verified-label accumulation	Different initial scoring mechanism
Costly-verification RL [Nijhuis and van Lelyveld, 2023]	Reward-driven information acquisition	Learns an acquisition policy; SAD uses fixed rules	SAD's entropy gate is not a reward policy
Claims graph features [Vorobyev, 2024]	Relational feature construction	Requires links beyond independent claim attributes	Adds relational information rather than SAD's entropy gate
SAD, this study	Anomaly votes, verification, supervised votes	Retrains once after each reviewed batch	Applies candidate-local EAD after initial consensus

3 METHODOLOGY

SAD combines predictor preprocessing, anomaly screening, supervised classification and batch-wise model updating. The following subsections specify these components, their integration and the evaluation measures.

3.1 PRE PROCESSING

Dimensionality reduction decreases the number of input variables while retaining useful characteristics of the data. In this research, the preprocessing step uses feature screening through a Gini criterion for categorical variables and a near-zero-variance criterion for numerical variables [Hastie et al., 2009; Kuhn, 2008].

Dimension Reduction

Gini Index: For a categorical predictor with category frequencies p_r, the Gini impurity is

$$Gini = 1 - \sum_{r=1}^{R} p_r^2.$$

It is zero for a constant feature and increases as the observations are distributed more evenly across categories. We retain predictors with $g_L < Gini < g_U$, where $g_L = 0.1$ and $g_U = 0.9$. This marginal variability screen does not measure association with fraud; it can discard informative high-cardinality features.

Near Zero Variance: Numerical predictors are removed if constant, or if their most-common to second-most-common value frequency ratio exceeds 19 and their percentage of unique values is at most 10%, following the near-zero-variance screening convention [Kuhn, 2008].

To prevent information from later evaluation batches entering fitted preprocessing, partitioning precedes feature screening. The screen, missing value handling, scaling and categorical encoding are fitted using D_1 predictors only, as made explicit in Algorithm 1.

Algorithm 1 PreProcessing Algorithm

Require: Dataset D; partition fractions $p_1, \ldots, p_m$; Gini bounds g_L, g_U
Ensure: Partitions $D_1, \ldots, D_m$ and fixed predictor transformations
1: Withhold outcomes from predictors; retain original row identities.
2: Partition records with sizes proportional to $p_1, \ldots, p_m$.
3: On D_1 only, remove categorical predictors outside (g_L, g_U) in Gini impurity.
4: On D_1 only, remove numerical predictors satisfying near-zero variance.
5: Fit imputation, scaling and encoding on retained D_1 predictors.
6: Apply those fixed transformations to every partition D_j.

Policy identifiers, postal identifiers and incident addresses are excluded. Near-unique categorical fields are also excluded. The two date fields in Insurance Claims are expanded into year, month and weekday; this does not make the experimental split chronological. The marker `?` is treated as missing. Numerical medians and categorical modes provide imputation; scaling and one-hot encoding use D_1 parameters and levels, with unseen categories mapped to zero indicators. Outcomes are removed by name before any detector or transformation is fitted, and row identities are preserved.

Balancing

Resampling structure: We use the Synthetic Minority Over-sampling Technique (SMOTE) to address class imbalance during supervised training [Chawla et al., 2002; He and Garcia, 2009]. SMOTE constructs additional minority-class training vectors along line segments joining a minority observation to a nearby observation from the same class. In the revised experiments, SMOTE is applied only to accumulated verified training records, never to future evaluation observations. The neighbor count is at most five and is reduced for small minority sets. One minority observation invokes random oversampling; a single-class set raises an error. SMOTE operates on encoded features, so synthetic one-hot values may be fractional. These are training vectors, not synthetic policies. Previously generated synthetic records are not accumulated across rounds.

Structure

The two SAD workflows are illustrated below. Figure 1 connects preprocessing and data partitioning to initial anomaly screening, verification and supervised learning. In subsequent iterations, the trained models identify potential fraud in the next subset, followed by expert confirmation and model updating. Figure 2 shows the multilayered structure, where EAD is applied to the result of anomaly consensus before expert verification. Figures 1 and 2 provide conceptual overviews of the retained workflows. Algorithms 1–3 specify their implementation: preprocessing is fitted on D_1 after partitioning; expert review includes alerts and a random sample of non-alerts; both fraud and genuine verified records enter training (the diagram label "Frauds" is shorthand for the reviewed records); and no model is retrained after the last evaluation batch. The terminal trained model in the conceptual diagrams is relevant only if further batches will arrive. Simple-A uses nine initial detector votes; Simple-B adds entropy as a tenth vote. In Multilayer, EAD gates the initial candidate subset before verification.

ANOMALY DETECTION

Before verified labels become available, the initial ensemble screens claims for departures from empirical feature patterns. An anomaly flag indicates a case for review, not a confirmed fraud [Chandola et al., 2009]. The nine

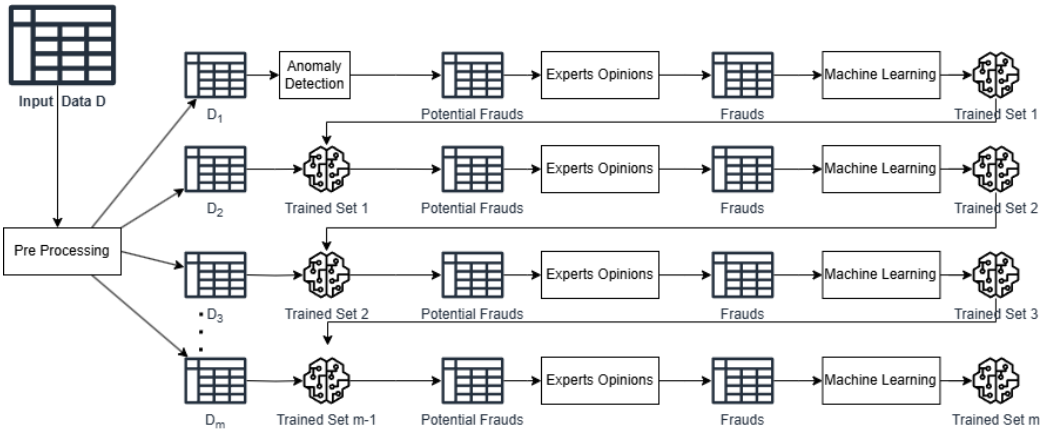


Figure 1: Simple SAD workflow: initial screening, verification and model updating across successive batches.

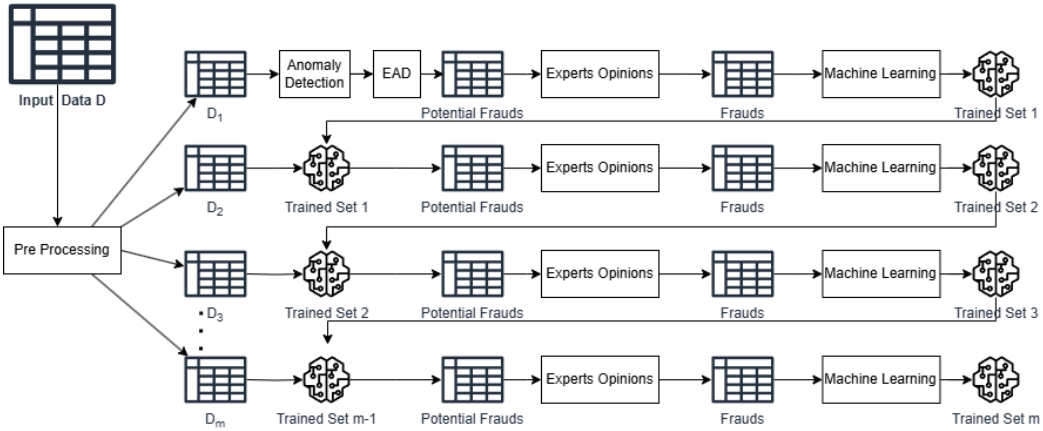


Figure 2: Multilayered SAD workflow, with entropy screening between initial anomaly consensus and verification.

retained detectors are specified below.

Box Plot: Observations outside $[Q_1 - 1.5IQR, Q_3 + 1.5IQR]$, where $IQR = Q_3 - Q_1$, are flagged using the boxplot rule. The rule is applied separately to retained numerical features, and the union of flagged records supplies one detector vote.

Percentile structure: Percentiles divide an empirical distribution according to its cumulative frequency. Values below the fifth percentile or above the ninety-fifth percentile are flagged in each numerical feature; the

union provides one vote.

Z-score structure: The Z score compares an observation with a feature's mean and standard deviation. The prescribed rule flags $|Z| > 3$. Its interpretation depends on the feature distribution.

Hampel filter structure: Robust location and scale reduce the influence of extreme values [Hampel, 1974]. Here the batch-wise rule flags a numerical value when its distance from the median exceeds $3 \times 1.4826 \times MAD$. It is not a sliding temporal filter.

Grubbs test structure: Grubbs' test compares an extreme standardized deviation with a sample-size-dependent critical value under a normality assumption [Grubbs, 1969]. We use the two-sided critical threshold at $\alpha = 0.05$ as a screening rule. This is not an iterative multiple-outlier hypothesis-testing procedure.

KNN structure: Distance-based outlier methods assess how far an observation lies from neighboring observations [Ramaswamy et al., 2000]. The implemented score is the mean distance to five neighbors in standardized numerical space; the top 20% are flagged. This averaging is an implementation choice, rather than an exact reproduction of the cited kth-distance rule.

Local outlier factor (LOF) structure: LOF quantifies how sparse an observation's neighborhood is relative to the neighborhoods of nearby observations [Breunig et al., 2000]. The implementation uses 20 neighbors and a contamination setting of 0.15.

Isolation forest structure: Isolation Forest constructs randomized partition trees and scores observations using their isolation paths [Liu et al., 2008]. We use 200 trees, subsamples of at most 256 records and contamination 0.10 on the encoded predictor matrix.

K Modes structure: K-Modes extends centroid-based clustering to categorical observations using category matching and modal representatives [Huang, 1998]. We use up to 30 clusters, one Huang initialization and 20 iterations, flagging the five smallest nonempty clusters. Small cluster membership is a screening heuristic, not evidence of fraud by itself.

Entropy Anomaly Detection (EAD): The information content of a categorical feature is measured by Shannon entropy [Shannon, 1948]. The original removal-based EAD idea compares the entropy of a working dataset with its entropy after one observation is removed. For subset S and predictor

ℓ, let $H_\ell(S) = -\sum_r p_{\ell r} \log_2 p_{\ell r}$. The exact implemented score is

$$Gain(x_i; S) = \frac{1}{p} \sum_{\ell=1}^{p} \{H_\ell(S) - H_\ell(S \setminus \{x_i\})\}. \quad (1)$$

Numerical variables use up to ten quantile bins fitted on S; those bins remain fixed during leave-one-out calculation. Frequency counts permit exact scores without rebuilding the dataset for every removal. The highest 20% of scores form the entropy shortlist. Simple-B computes EAD on D_1; Multilayer computes it on the anomaly-consensus subset Y_1.

Threshold limits: The anomaly threshold T_{AD} specifies the minimum number of positive detector votes required to flag an observation. A record is flagged once its positive-vote count reaches T_{AD}. The threshold operates on detector consensus.

3.2 MACHINE LEARNING

Following anomaly detection, expert opinions supply training labels for supervised learning. The ensemble includes the following eight model families already used in the thesis implementation.

LR models: Logistic regression estimates a binary outcome probability from the input features [Hastie et al., 2009].

SVM models: A support-vector classifier constructs a separating hyperplane; kernel extensions can model nonlinear boundaries [Cortes and Vapnik, 1995]. The present ensemble uses a linear kernel.

XGB models: XGBoost builds an additive ensemble of decision trees with regularized gradient boosting [Chen and Guestrin, 2016].

RF models: Random forests combine trees trained with randomized observations and feature selections; their collective predictions form an ensemble estimate [Breiman, 2001].

NB models: Naive Bayes combines class-conditional feature models under a conditional-independence assumption [Domingos and Pazzani, 1997]. The implementation uses Gaussian feature likelihoods.

KNN models: Nearest-neighbor classification uses labeled examples close to the observation being classified [Cover and Hart, 1967]. This supervised component is distinct from the initial KNN outlier detector.

AdaBoost models: AdaBoost combines weak learners through successive reweighting of training observations [Freund and Schapire, 1997].

Table 3: Fixed specifications of the original eight supervised model families.

Model	Specification
LR	L2 logistic regression; liblinear; $C = 1$; 1,000 iterations
SVM	Linear kernel; $C = 1$; training-only probability estimation
XGBoost	120 trees; depth 4; learning rate .05; row/column subsampling .8
RF	200 trees; minimum leaf size 2
NB	Gaussian conditional feature model
KNN	Five neighbors; distance weighting
AdaBoost	100 decision stumps; learning rate .5
ANN	32 hidden units; L2 penalty 10^{-4}; at most 200 iterations; early stopping with 15% internal validation

ANN models: Artificial neural networks learn weights between interconnected layers to model relationships between predictors and outcomes [Rumelhart et al., 1986]. We use one hidden layer.

Threshold limit: The supervised threshold T_{ML} specifies the minimum number of positive model votes for an alert. For learner k, $V_k(x) = \mathbb{1}\{\hat{p}_k(x) \geq 0.5\}$, and $\hat{y}(x) = \mathbb{1}\{\sum_{k=1}^{M} V_k(x) \geq T_{ML}\}$. The mean score $M^{-1} \sum_k \hat{p}_k(x)$ is used for ranking metrics. After oversampling, that score need not be a calibrated population fraud probability.

3.3 Sequential Anomaly Detection (SAD)

In the following, we describe two structures using the partitions returned by Algorithm 1. The inputs include the anomaly detector set $AD\ SET$, the supervised model set $ML\ SET$, their sizes N and M, vote thresholds T_{AD} and T_{ML}, expert opinions and the non-alert sampling percentage PN.

Throughout the algorithms, "potential normal" means *not alerted*, not verified genuine. In batch j, F_j is the alert set and G_j is a random sample of $\lceil (PN/100)|D_j \setminus F_j| \rceil$ non-alerts. Both sets are submitted for verification before their labels are available to the learning algorithm. E_j denotes these newly verified records, $L_j = \bigcup_{r \leq j} E_r$ the accumulated real training records, and $E'_j = SMOTE(L_j)$ the current balanced training set. In the experiments, $PN = 10$. Stored outcomes simulate expert opinions; no actual expert investigation was conducted in this retrospective evaluation.

Simple structure of the SAD algorithm

The anomaly detection process starts with unlabeled data from the first partition. Every detector supplies a normal (0) or anomalous (1) vote for each observation. The total U_{Total} is compared with T_{AD} to identify potential frauds. Expert opinions then provide labels for training the supervised models. At the end of each subsequent partition, the models are updated with newly verified observations.

The balanced set E_1' trains each model in $ML\ SET$, forming TS_1. All observations in D_2 are then classified using those trained models. Each model contributes one vote; if their sum reaches T_{ML}, the observation is flagged for expert review. Balancing and model updating are repeated for subsequent partitions as stated in Algorithm 2. Simple-A uses the nine anomaly detectors above. Simple-B adds EAD as a tenth vote in the same initial consensus. Their common integer threshold therefore corresponds to different vote fractions; this architecture comparison does not hold the initial review budget exactly constant.

Multilayered structure of the SAD algorithm

The multilayered structure also starts with anomaly-detector votes on the first partition. Their sum identifies a candidate set Y_1. These candidates undergo re-examination using EAD, and only those also selected by EAD enter the alert set F_1. Expert verification supplies the labels used for balancing and supervised training. Subsequent partitions use the same voting, verification and updating cycle as the simple structure.

Thus the extra layer changes the initial acquisition set, rather than merely adding another learner to the final supervised ensemble. The entropy bins and scores are computed within Y_1; screening the candidate subset is not equivalent to applying the full-D_1 entropy ranking unchanged. Algorithm 3 preserves the original candidate-to-entropy ordering and makes this distinction explicit.

The batch order is sequential in information access, not necessarily in calendar time. Every prediction in D_j is saved before E_j is revealed. Updating occurs once after the complete batch, outside the row-wise voting loop. No update is required after D_m because there is no later evaluated prediction. These specifications correct ambiguities in the earlier pseudocode while retaining its anomaly–verification–learning architecture.

Algorithm 2 Simple structure of the SAD algorithm

Require: D; $AD\ SET = (AD_1, \ldots, AD_N)$; $ML\ SET = (ML_1, \ldots, ML_M)$;
thresholds T_{AD}, T_{ML}; expert verification; sampling percentage PN
Ensure: Trained models and potential frauds in each partition
Preprocess and partition D using Algorithm 1.
Apply $AD\ SET$ to D_1 to obtain binary votes $U_1(x), \ldots, U_N(x)$.
for each row x of D_1 **do**
    Set $U_{Total}(x) = \sum_{k=1}^{N} U_k(x)$.
    if $U_{Total}(x) \geq T_{AD}$ **then**
        Label x as Potential Fraud.
    else
        Label x as Potential Normal.
    end if
end for
Create F_1 as all Potential Fraud rows in D_1.
Sample G_1 randomly from $D_1 \setminus F_1$ at percentage PN.
Verify both F_1 and G_1; receive labeled set E_1; set $L_1 = E_1$.
Create balanced training data $E_1' = SMOTE(L_1)$.
Train all models in $ML\ SET$ on E_1'; denote them TS_1.
for $j = 2, \ldots, m$ **do**
    Use TS_{j-1} to predict all of D_j before verification.
    for each row x of D_j **do**
        Store model votes $V_1(x), \ldots, V_M(x)$ and sum $V_{Total}(x)$.
        If $V_{Total}(x) \geq T_{ML}$, label Potential Fraud; otherwise Potential Normal.
    end for
    Create alert set F_j and random non-alert sample G_j at percentage PN.
    Save all predictions; verify $F_j \cup G_j$ to obtain E_j.
    if $j < m$ **then**
        Set $L_j = L_{j-1} \cup E_j$ using only real verified records.
        Create $E_j' = SMOTE(L_j)$; retrain all models to obtain TS_j.
    end if
end for

3.4 EVALUATION

The original evaluation criteria are accuracy, recall and F1. We explicitly define fraud ($y = 1$) as the positive class and add precision, specificity, balanced accuracy and ranking metrics because accuracy alone can obscure minority-class performance [Saito and Rehmsmeier, 2015; He and Garcia, 2009]. Let TP, TN, FP, FN denote the confusion counts for that class.

Accuracy metric: Accuracy is the fraction of correctly classified observations:

$$Acc = \frac{TP + TN}{TP + TN + FP + FN}.$$

Algorithm 3 Multilayered structure of the SAD algorithm

Require: D; nine-detector $AD\ SET$; $ML\ SET$; T_{AD}, T_{ML};
expert verification; sampling percentage PN; entropy retention fraction q
Ensure: Trained models and potential frauds in each partition
Preprocess and partition D using Algorithm 1.
Apply the $N = 9$ detectors to D_1; initialize $Y_1 = \varnothing$.
for each row x of D_1 **do**
  Set $U_{Total}(x) = \sum_{k=1}^{N} U_k(x)$.
  if $U_{Total}(x) \geq T_{AD}$ **then**
    Add x to candidate set Y_1.
  end if
end for
Compute candidate-local $Gain(x; Y_1)$ using Equation 1.
Set $C_1(x) = 1$ for the highest q fraction of scores within Y_1.
Create $F_1 = \{x \in Y_1 : C_1(x) = 1\}$; all other rows are Potential Normal.
Sample G_1 randomly from $D_1 \setminus F_1$ at percentage PN.
Verify $F_1 \cup G_1$; receive labeled E_1 and set $L_1 = E_1$.
Create $E_1' = SMOTE(L_1)$; train $ML\ SET$ to obtain TS_1.
for $j = 2, \ldots, m$ **do**
  Use TS_{j-1} to predict all of D_j before verification.
  for each row x of D_j **do**
    Store model votes $V_1(x), \ldots, V_M(x)$ and sum $V_{Total}(x)$.
    If $V_{Total}(x) \geq T_{ML}$, label Potential Fraud; otherwise Potential Normal.
  end for
  Create alert set F_j and random non-alert sample G_j at percentage PN.
  Save all predictions; verify $F_j \cup G_j$ to obtain E_j.
  if $j < m$ **then**
    Set $L_j = L_{j-1} \cup E_j$ using only real verified records.
    Create $E_j' = SMOTE(L_j)$; retrain $ML\ SET$ to obtain TS_j.
  end if
end for

F1-score metric: With precision $PS = TP/(TP + FP)$ and recall $RS = TP/(TP + FN)$, the harmonic mean is

$$F1 = \frac{2PS\,RS}{PS + RS}.$$

Recall/Sensitivity metric: Recall RS measures the fraction of recorded frauds correctly flagged by the model.

Specificity is $TN/(TN+FP)$ and balanced accuracy is the mean of specificity and recall. ROC–AUC uses the continuous score. Average precision (AP), stored as `pr_auc` in the exports, is the stepwise precision–recall area $\sum_k (R_k - R_{k-1})P_k$, not trapezoidal interpolation. Zero denominators in pre-

cision/F1 are assigned zero. Full held-out stored outcomes are used only for retrospective scoring, including outcomes not acquired for training.

The alert rate is $|F_j|/|D_j|$; the review rate is $(|F_j| + |G_j|)/|D_j|$. Final-batch review is counted as workload, although its outcomes train no evaluated model. We pool D_2–D_4 within each seed and report means and sample standard deviations over ten seeds. These repeated splits reuse observations: their standard deviations describe split variability and are not independent-sample confidence intervals.

AI assistance. ChatGPT (OpenAI) assisted with manuscript editing and revision of the analysis and plotting scripts. The authors retain responsibility for the code, results and final text.

4 Real Data Application

The experiments apply the SAD components defined in Section 3 to two insurance-claims benchmarks. The evaluation compares the retained architectures, individual classifiers, a frozen-model control and two joint threshold settings.

Carclaims dataset

Carclaims contains 15,420 records, 32 original predictors and 923 recorded frauds (5.99%) [Bosco, 2020]. The original descriptive frequency table is retained and regenerated from the supplied data in Table 4.

4.1 Additional dataset and replication scope

Insurance Claims is available through the Kaggle dataset published by `mexwell` [mexwell, nd], the source identified for the second file supplied in the research archive. The analyzed file contains 1,000 records, 38 original predictors and 247 recorded frauds (24.70%). Its schema, record count and contents differ from Carclaims. The differently named `fraud_oracle.csv` is a duplicate representation of Carclaims and is excluded from the additional benchmark.

Because the feature spaces differ, preprocessing and models are fitted separately for each dataset under the same SAD protocol. This experiment

Table 4: Frequency table for selected categorical variables in Carclaims. Original descriptive variables are retained; percentages are recomputed from the supplied file.

Feature	Category	Percent	Frequency
Sex	Female	15.69	2,420
	Male	84.31	13,000
MaritalStatus	Divorced	0.49	76
	Married	68.90	10,625
	Single	30.38	4,684
	Widow	0.23	35
VehicleCategory	Sedan	62.72	9,671
	Sport	34.75	5,358
	Utility	2.54	391
DriverRating	1	25.58	3,944
	2	24.65	3,801
	3	25.19	3,884
	4	24.58	3,791
FraudFound	No	94.01	14,497
	Yes	5.99	923

Table 5: Input datasets before predictor screening.

Dataset	Records	Predictors	Recorded fraud
Carclaims	15,420	32	923 (5.99%)
Insurance Claims	1,000	38	247 (24.70%)

assesses replication of the workflow across benchmarks. The Kaggle source documents access to the second dataset; the original collecting insurer and its full real-versus-synthetic provenance remain unverified.

4.2 Algorithm specifications

The number of partitions is $m = 4$, with proportions $p_1 = 0.50$, $p_2 = 0.30$, $p_3 = 0.10$ and $p_4 = 0.10$. Simple-A uses $N = 9$ initial detectors; Simple-B uses $N = 10$ by including EAD as a vote; Multilayer retains $N = 9$ and applies EAD to Y_1. All three retain $M = 8$ supervised learners: LR, SVM, XGB, RF, NB, KNN, AdaBoost and ANN. Seeds 1234–1243 and the four-part schedule are shared across variants. The main operating point is $(T_{AD}, T_{ML}) = (3, 4)$; the stricter point $(5, 6)$ is a complete additional Mul-

tilayer training trajectory. These are fixed illustrative settings, not a preregistered optimum. The data had been examined during development; the second benchmark is not described as prospectively untouched external data. SAD hyperparameters are unchanged between datasets. Only the PSO comparator follows the predefined inner training-only search. Random partitions are stratified using the outcomes for split construction only. Within each trajectory, label access follows Algorithms 2 and 3.

Reconciliation with the original exports. All reported performance values are recomputed from the corrected implementation. The revision uses training-only preprocessing and SMOTE, corrected entropy/index alignment, and outcome access only after review selection. Non-alert samples are verified rather than assumed genuine. The previous exports did not consistently designate fraud as the positive class. Consequently, their reported recall/F1 and high overall accuracy are not reused as evidence of fraud detection performance. These corrections are necessary to evaluate the retained SAD architectures consistently.

4.3 Architecture performance and workload

Table 6: Fraud-class performance and total review fraction; mean ± sample SD across ten seeds. All metrics pool D_2–D_4 within a seed. AP denotes average precision.

Architecture	Precision	Recall	F1	AP	Review
Carclaims					
Simple-A	0.144 ± 0.009	0.480 ± 0.039	0.221 ± 0.014	0.151 ± 0.009	0.280 ± 0.012
Simple-B	0.146 ± 0.006	0.489 ± 0.044	0.224 ± 0.009	0.153 ± 0.008	0.281 ± 0.016
Multilayer	0.144 ± 0.008	0.407 ± 0.047	0.212 ± 0.014	0.140 ± 0.007	0.253 ± 0.016
Multilayer-static	0.141 ± 0.008	0.427 ± 0.046	0.211 ± 0.013	0.135 ± 0.009	0.264 ± 0.017
Insurance Claims					
Simple-A	0.491 ± 0.015	0.586 ± 0.078	0.533 ± 0.038	0.487 ± 0.029	0.368 ± 0.033
Simple-B	0.510 ± 0.038	0.624 ± 0.038	0.560 ± 0.028	0.491 ± 0.039	0.377 ± 0.028
Multilayer	0.465 ± 0.035	0.544 ± 0.101	0.497 ± 0.043	0.452 ± 0.048	0.365 ± 0.050
Multilayer-static	0.435 ± 0.042	0.546 ± 0.104	0.479 ± 0.041	0.442 ± 0.034	0.386 ± 0.067

On Carclaims, Simple-B has the highest mean F1 among the four architecture variants (0.224). Simple-B and updating Multilayer attain AP 0.153

Table 7: Complementary metrics under the same main protocol. High accuracy alone is insufficient for these imbalanced datasets.

Architecture	Specificity	Balanced acc.	ROC-AUC	Accuracy
Carclaims				
Simple-A	0.818 ± 0.013	0.649 ± 0.017	0.777 ± 0.010	0.797 ± 0.011
Simple-B	0.818 ± 0.016	0.653 ± 0.016	0.777 ± 0.008	0.798 ± 0.013
Multilayer	0.846 ± 0.016	0.626 ± 0.018	0.752 ± 0.017	0.819 ± 0.013
Multilayer-static	0.834 ± 0.018	0.631 ± 0.017	0.744 ± 0.018	0.810 ± 0.015
Insurance Claims				
Simple-A	0.800 ± 0.025	0.693 ± 0.028	0.738 ± 0.015	0.747 ± 0.009
Simple-B	0.800 ± 0.035	0.712 ± 0.020	0.742 ± 0.019	0.756 ± 0.024
Multilayer	0.792 ± 0.045	0.668 ± 0.035	0.707 ± 0.026	0.730 ± 0.020
Multilayer-static	0.761 ± 0.070	0.654 ± 0.032	0.694 ± 0.024	0.708 ± 0.035

and 0.140, respectively; Multilayer's mean F1 is 0.212. The corresponding review fractions are 0.281 for Simple-B and 0.253 for Multilayer.

On Insurance Claims, Simple-B has the highest mean F1 among the four architecture variants (0.560). Simple-B and updating Multilayer attain AP 0.491 and 0.452, respectively; Multilayer's mean F1 is 0.497. The corresponding review fractions are 0.377 for Simple-B and 0.365 for Multilayer.

These differences concern both predictive performance and the records acquired for training. The architectures are therefore compared as complete review policies, without holding the review budget constant. Figures 3–5 summarize pooled performance, partition-wise detection and verification workload.

4.4 Classifier comparisons and ranking curves

The eight component classifiers are evaluated individually on the same verified training records acquired by Simple-B. Two additional comparators use those same records: balanced random forest (200 trees with within-tree balancing and no preceding SMOTE) and PSO-XGBoost adapted from the insurance fraud study of Ding et al. [2025]. The latter is a recent fraud-motivated comparator; it is not an exact reproduction of the published system.

PSO selects hyperparameters once using only verified D_1 records and three-fold stratified cross-validation scored by AP. SMOTE is fitted inside each training fold. Four particles and three iterations search 60–160 trees, depth 2–6, learning rates 10^{-2}–$10^{-0.7}$ and row/column subsampling 0.6–1.

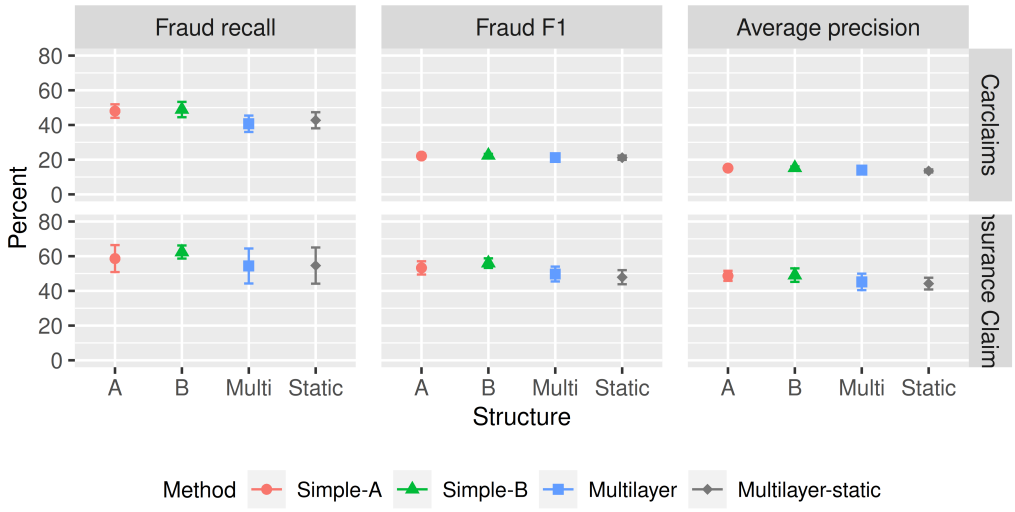


Figure 3: Fraud recall, F1 and AP for four architectures. Points and bars show mean and sample SD across ten random splits, pooling D_2–D_4 within each seed.



Figure 4: Partition-wise F1 and recall. Later partitions contain different records; these are not learning curves on a fixed test set.

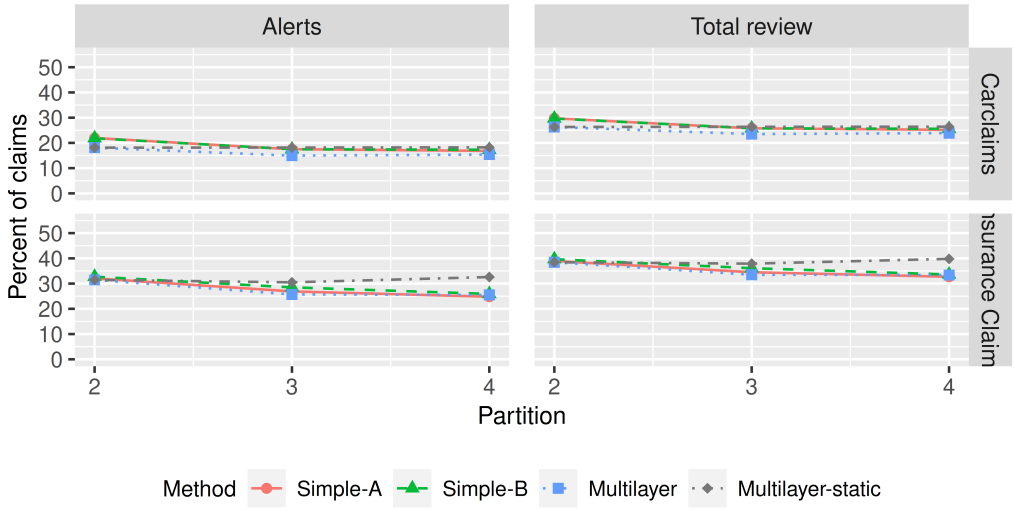


Figure 5: Partition-wise alert rate and total review rate, including the 10% audit of non-alerted claims. Lines and points show means over ten random splits. The panels distinguish alerted claims from the full set submitted for verification.

Inertia is 0.7 and both attraction coefficients are 1.4. Selected values are fixed in subsequent rounds. The comparison tests classifier choice under a common acquisition policy, not ten independent end-to-end acquisition systems. For Carclaims, BalancedRF leads this comparator set in both mean F1 (0.231) and AP (0.168). For Insurance Claims, PSO-XGB-adapted leads this comparator set in both mean F1 (0.599) and AP (0.522). These rankings are conditional on Simple-B's acquisition policy and the stated training settings. Differences in tuning budgets limit comparisons with fully optimized implementations of the published methods. Figures 6 and 7 show the classifier comparison and the architectures' ranking performance on the predefined split.

4.5 Ablation study and threshold sensitivity

Comparing Simple-A with Simple-B examines adding a flat entropy vote; comparing Simple-B with Multilayer examines replacing flat entropy voting with candidate-local entropy gating. These comparisons also change which records are reviewed and therefore are not matched-budget causal estimates.

Table 8: Conditional classifier comparison using Simple-B's verified records. The two columns per benchmark report fraud F1 and AP. PSO-XGB is the explicitly adapted comparator.

Method	Carclaims F1	Carclaims AP	Insurance F1	Insurance AP
LR	0.221 ± 0.007	0.143 ± 0.008	0.508 ± 0.021	0.461 ± 0.035
SVM	0.217 ± 0.008	0.137 ± 0.007	0.501 ± 0.037	0.455 ± 0.036
XGB	0.076 ± 0.017	0.159 ± 0.009	0.551 ± 0.033	0.497 ± 0.036
NB	0.155 ± 0.022	0.101 ± 0.016	0.500 ± 0.022	0.460 ± 0.022
RF	0.039 ± 0.015	0.161 ± 0.006	0.537 ± 0.032	0.511 ± 0.046
KNN	0.172 ± 0.008	0.101 ± 0.005	0.413 ± 0.013	0.296 ± 0.026
AdaBoost	0.216 ± 0.012	0.144 ± 0.006	0.558 ± 0.030	0.512 ± 0.036
ANN	0.188 ± 0.011	0.140 ± 0.010	0.404 ± 0.033	0.360 ± 0.046
BalancedRF	0.231 ± 0.012	0.168 ± 0.007	0.595 ± 0.025	0.497 ± 0.029
PSO-XGB-adapted	0.116 ± 0.073	0.158 ± 0.010	0.599 ± 0.026	0.522 ± 0.032
Simple-B	0.224 ± 0.009	0.153 ± 0.008	0.560 ± 0.028	0.491 ± 0.039

For the sequential ablation, Multilayer-static freezes the exact D_1 models and predicts the same D_2–D_4 records without retraining. Its initialization, preprocessing and initial training set are identical to those of updating Multilayer. For Carclaims, sequential updating changes mean F1 by +0.0005,

Table 9: Paired seed-wise change from static to updating Multilayer: mean ± sample SD. These are descriptive paired differences, not confidence intervals.

Dataset	Δ F1	Δ AP	Δ review
Carclaims	0.0005 ± 0.0081	0.0049 ± 0.0052	-0.0109 ± 0.0070
Insurance Claims	0.0182 ± 0.0223	0.0098 ± 0.0211	-0.0216 ± 0.0304

AP by +0.0049 and review fraction by -0.0109 relative to the frozen model. For Insurance Claims, sequential updating changes mean F1 by +0.0182, AP by +0.0098 and review fraction by -0.0216 relative to the frozen model. The Carclaims F1 change is small relative to its split-to-split variation. These descriptive results do not establish statistical superiority. Because updating affects later acquisition as well as prediction, the ablation measures the complete update policy. The two threshold settings are evaluated through separate complete trajectories.

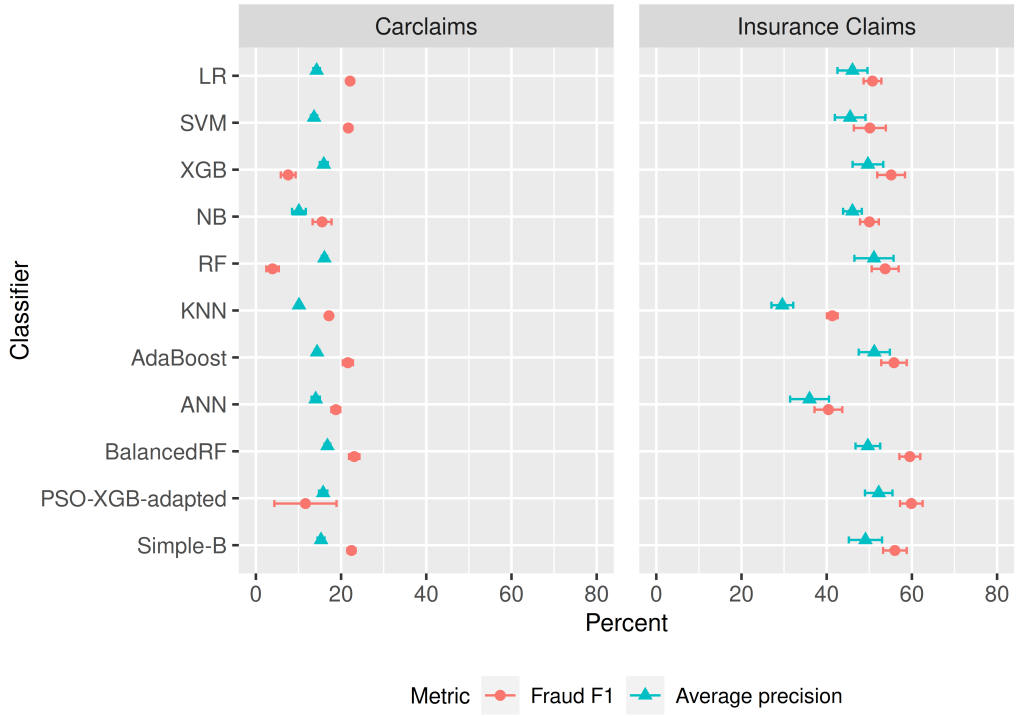


Figure 6: Same-acquisition classifier comparison. All comparators use the Simple-B verified-record policy. PSO-XGB is an adapted search; BalancedRF balances within trees. Error bars show sample SD.

Table 10: Multilayer sensitivity at two joint threshold settings; mean ± sample SD.

Dataset / thresholds	Recall	F1	AP	Review
Carclaims / (3,4)	0.407 ± 0.047	0.212 ± 0.014	0.140 ± 0.007	0.253 ± 0.016
Carclaims / (5,6)	0.111 ± 0.029	0.123 ± 0.021	0.127 ± 0.007	0.142 ± 0.010
Insurance Claims / (3,4)	0.544 ± 0.101	0.497 ± 0.043	0.452 ± 0.048	0.365 ± 0.050
Insurance Claims / (5,6)	0.300 ± 0.120	0.369 ± 0.118	0.440 ± 0.059	0.231 ± 0.051

4.6 Computation and reproducibility

Predictions retain original row ID, split, seed, architecture, score, votes, alert, review mask and recorded outcome. Manifests record software versions and code/data hashes. Numerical verification recomputes metrics from these

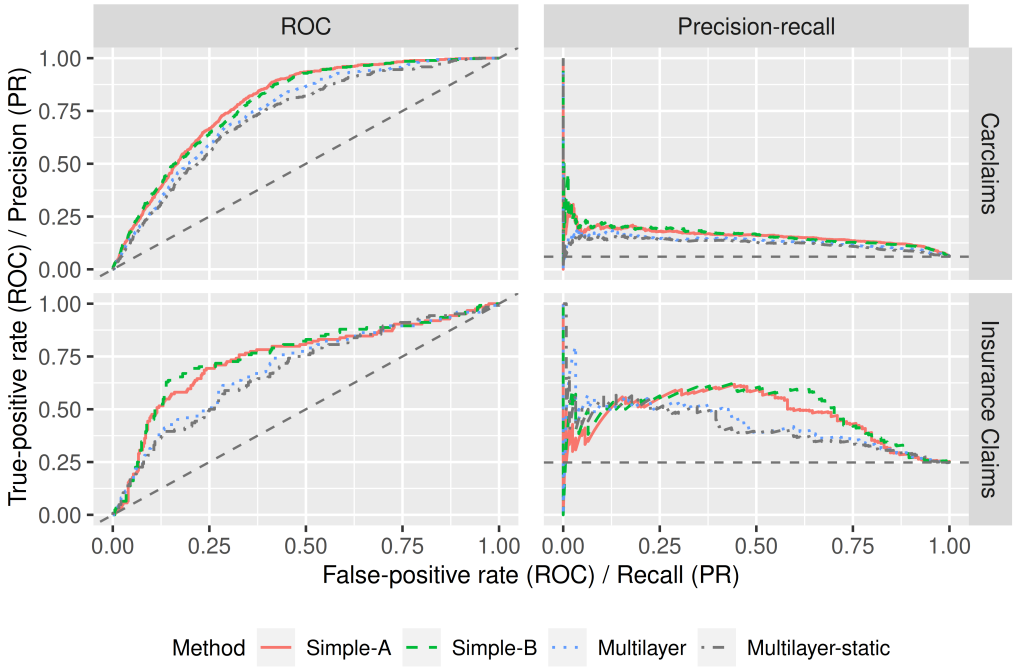


Figure 7: ROC and precision–recall curves for predefined seed 1234, pooling D_2–D_4. These are representative split curves, not averages over independent populations. Dashed lines are no-skill references.

records; figures and LaTeX tables use the same results. The executed implementation is Python, using scikit-learn and the associated packages listed in the environment record [Pedregosa et al., 2011]. Original R scripts and exports are archived; revised results come from Python. A single R/`ggplot2` script renders both benchmarks from the saved results. The empirical figures use the same visual conventions and share axis limits within each comparison. Performance axes span 0–80% and workload axes 0–55%, encompassing the displayed values and error bars; ROC and precision–recall curves retain the full 0–1 range. Numerical tables report fractions on the 0–1 scale. Error bars denote sample standard deviations across splits, not confidence intervals.

Data and code availability. The source datasets are identified in Bosco [2020] and mexwell [nd]. The analyzed files, data dictionaries, executable analysis and plotting scripts, prediction-level outputs and instructions for re-

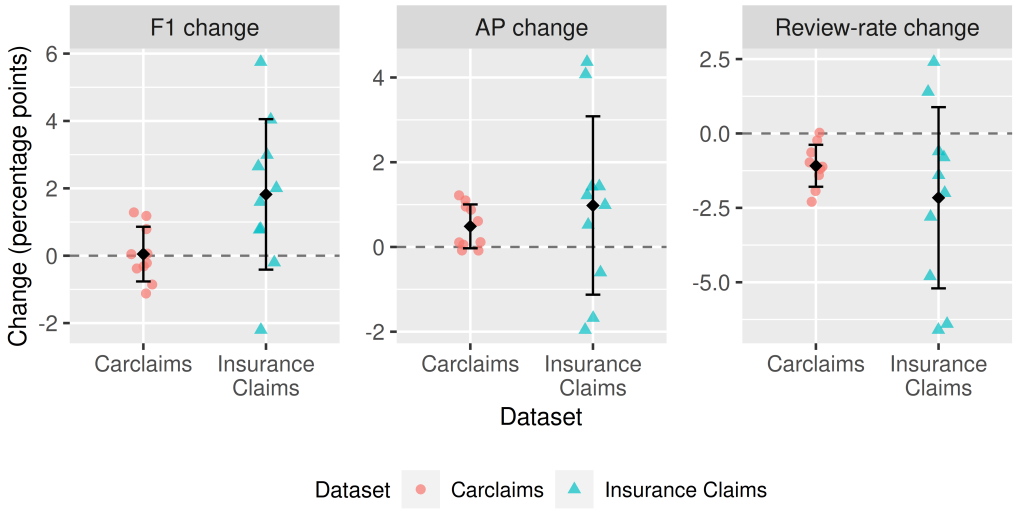


Figure 8: Paired seed-wise changes, in percentage points, from frozen to updating Multilayer. Positive F1/AP changes favor updating; review-rate differences measure changed workload. Black diamonds and bars show mean and sample SD. Panels use separate vertical ranges.

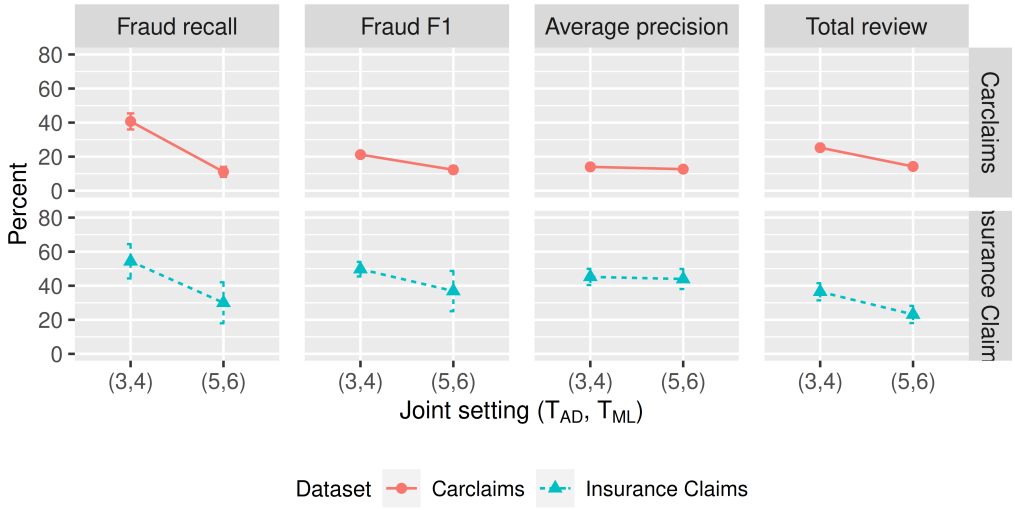


Figure 9: Main $(3, 4)$ and strict $(5, 6)$ Multilayer points from separately rerun trajectories. The comparison does not isolate the two thresholds independently.

Table 11: Wall time in seconds, mean ± sample SD. Architecture rows sum supervised fitting at D_1, D_2, D_3; the shared anomaly stage is reported separately and excludes preprocessing.

Operation	Carclaims	Insurance Claims
Simple-A	15.94 ± 0.79	1.69 ± 0.10
Simple-B	17.67 ± 1.21	1.83 ± 0.09
Multilayer	10.84 ± 0.79	1.30 ± 0.06
Shared anomaly stage	2.38 ± 0.23	0.39 ± 0.03

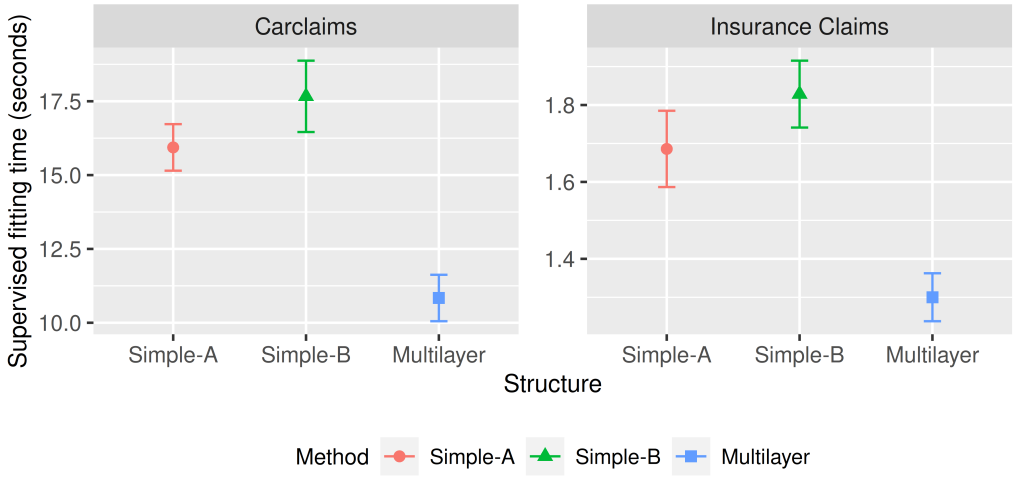


Figure 10: Supervised fitting time across three training rounds, excluding shared initial anomaly calculation, exports and comparator tuning. Times describe this hardware and implementation; the panels use different time ranges.

producing the tables and figures are publicly available in the project GitHub repository.[2] The archived analysis uses the supplied dataset versions; source hashes are included to identify these files unambiguously.

[2] https://github.com/Mehrab-Atighi/Anomaly-Detection-Algorithm-for-Enhanced-Fraud-Identification-in-the-Insurance-Sector

5 CONCLUSION

This study presents SAD for insurance fraud screening. The original framework combines anomaly detection, expert verification and supervised learning in successive partitions. Its simple and multilayered structures share the update cycle but differ in the use of entropy during initial candidate selection. Algorithms 1–3 describe these structures explicitly.

The revised experiments compare the three architectural variants on Carclaims and a second Insurance Claims benchmark, add a frozen-model ablation and broaden the supervised comparisons. On the second benchmark, Simple-B attains mean fraud F1 0.560 and AP 0.491; updating Multilayer attains F1 0.497 at review fraction 0.365. These values demonstrate an empirical workload–detection trade-off rather than a universal benefit of adding an entropy gate. Architecture selection should therefore consider detection performance together with the available investigation capacity.

Limitations and future research. The partitions are retrospective and random, not chronological. Stored labels simulate expert decisions without investigation delay, errors or monetary cost. Random audits provide verified negatives and some missed frauds, but do not eliminate acquisition bias. The second benchmark has a documented Kaggle access source but unverified collecting-insurer provenance. It supports replication under a different schema and prevalence; prospective validation at an independent insurer remains necessary. Its smaller size also limits generalization. AP values across different prevalences should not be interpreted as directly comparable measures of task difficulty.

The feature screen can discard useful variables; SMOTE on encoded categorical predictors can produce fractional values; and ranking scores need not be calibrated probabilities. The classifier comparisons share the Simple-B acquisition policy. Recent deep and graph-based insurance systems requiring additional relationships or proprietary data have not been reproduced. The adapted PSO comparator and two threshold settings do not constitute exhaustive state-of-the-art benchmarking or optimization.

Future research should evaluate chronological claims with documented provenance, actual review costs, matched investigation budgets, categorical-aware resampling, calibration and prospectively fixed model selection. The accompanying code, data, prediction records and figure sources support reproduction of the results reported here.

References

Banulescu-Radu, D. and Yankol-Schalck, M. (2024). Practical guideline to efficiently detect insurance fraud in the era of machine learning: A household insurance case. *Journal of Risk and Insurance*, 91(4):867–913.

Bolton, R. J. and Hand, D. J. (2002). Statistical Fraud Detection: A Review. *Statistical Science*, 17(3):235–255.

Bosco, A. P. J. (2020). Auto insurance claims fraud detection. `https://www.kaggle.com/datasets/antopravinjohnbosco/auto-insurance-claims-fraud-detection`. Accessed 19 September 2026.

Breiman, L. (2001). Random Forests. *Machine Learning*, 45(1):5–32.

Breunig, M. M., Kriegel, H.-P., Ng, R. T., and Sander, J. (2000). LOF: identifying density-based local outliers. In *Proceedings of the 2000 ACM SIGMOD international conference on Management of data*, pages 93–104.

Chandola, V., Banerjee, A., and Kumar, V. (2009). Anomaly detection: A survey. *ACM Computing Surveys*, 41(3):1–58. Article 15.

Chawla, N. V., Bowyer, K. W., Hall, L. O., and Kegelmeyer, W. P. (2002). SMOTE: Synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research*, 16:321–357.

Chen, T. and Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 785–794.

Cortes, C. and Vapnik, V. (1995). Support-vector networks. *Machine Learning*, 20(3):273–297.

Cover, T. and Hart, P. (1967). Nearest neighbor pattern classification. *IEEE Transactions on Information Theory*, 13(1):21–27.

Debener, J., Heinke, V., and Kriebel, J. (2023). Detecting insurance fraud using supervised and unsupervised machine learning. *Journal of Risk and Insurance*, 90(3):743–768.

Ding, N., Ruan, X., Wang, H., and Liu, Y. (2025). Automobile insurance fraud detection based on PSO-XGBoost model and interpretable machine learning method. *Insurance: Mathematics and Economics*, 120:51–60.

Domingos, P. and Pazzani, M. (1997). On the Optimality of the Simple Bayesian Classifier under Zero-One Loss. *Machine Learning*, 29(2-3):103–130.

Freund, Y. and Schapire, R. E. (1997). A Decision-Theoretic Generalization of On-Line Learning and an Application to Boosting. *Journal of Computer and System Sciences*, 55(1):119–139.

Gomes, C., Jin, Z., and Yang, H. (2021). Insurance fraud detection with unsupervised deep learning. *Journal of Risk and Insurance*, 88(3):591–624.

Grubbs, F. E. (1969). Procedures for Detecting Outlying Observations in Samples. *Technometrics*, 11(1):1–21.

Hampel, F. R. (1974). The Influence Curve and its Role in Robust Estimation. *Journal of the American Statistical Association*, 69(346):383–393.

Hassani, H., Huang, X., Silva, E., and Ghodsi, M. (2020). Deep learning and implementations in banking. *Annals of Data Science*, 7:433–446.

Hastie, T., Tibshirani, R., and Friedman, J. (2009). *The Elements of Statistical Learning*. Springer, 2 edition.

He, H. and Garcia, E. A. (2009). Learning from Imbalanced Data. *IEEE Transactions on Knowledge and Data Engineering*, 21(9):1263–1284.

Hilal, W., Gadsden, S. A., and Yawney, J. (2022). Financial fraud: a review of anomaly detection techniques and recent advances. *Expert systems With applications*, 193:116429.

Huang, Z. (1998). Extensions to the k-Means Algorithm for Clustering Large Data Sets with Categorical Values. *Data Mining and Knowledge Discovery*, 2(3):283–304.

Kamalov, F., Thabtah, F., and Leung, H. H. (2023). Feature selection in imbalanced data. *Annals of Data Science*, 10:1527–1541.

Khalil, A. A., Liu, Z., Fathalla, A., Ali, A., and Salah, A. (2024). Machine learning based method for insurance fraud detection on class imbalance datasets with missing values. *IEEE Access*, 12:155451–155468.

Kuhn, M. (2008). Building Predictive Models in R Using the caret Package. *Journal of Statistical Software*, 28(5):1–26.

Liu, F. T., Ting, K. M., and Zhou, Z.-H. (2008). Isolation Forest. In *2008 Eighth IEEE International Conference on Data Mining*, pages 413–422.

mexwell (n.d.). Insurance claims. Kaggle. `https://www.kaggle.com/datasets/mexwell/insurance-claims`. Accessed 19 September 2026.

Moon, H., Pu, Y., and Ceglia, C. (2019). A predictive modeling for detecting fraudulent automobile insurance claims. *Theoretical Economics Letters*, 09:1886–1900.

Ngai, E., Hu, Y., Wong, Y., Chen, Y., and Sun, X. (2011). The application of data mining techniques in financial fraud detection: A classification framework and an academic review of literature. *Decision Support Systems*, 50(3):559–569.

Nian, K., Zhang, H., Tayal, A., Coleman, T., and Li, Y. (2016). Auto insurance fraud detection using unsupervised spectral ranking for anomaly. *The Journal of Finance and Data Science*, 2(1):58–75.

Nijhuis, M. and van Lelyveld, I. (2023). Outlier detection with reinforcement learning for costly to verify data. *Entropy*, 25(6):842.

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., and Duchesnay, É. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12(85):2825–2830.

Ramaswamy, S., Rastogi, R., and Shim, K. (2000). Efficient algorithms for mining outliers from large data sets. In *Proceedings of the 2000 ACM SIGMOD international conference on Management of data*, pages 427–438.

Rumelhart, D. E., Hinton, G. E., and Williams, R. J. (1986). Learning representations by back-propagating errors. *Nature*, 323(6088):533–536.

Saito, T. and Rehmsmeier, M. (2015). The Precision-Recall Plot Is More Informative than the ROC Plot When Evaluating Binary Classifiers on Imbalanced Datasets. *PLOS ONE*, 10(3):e0118432.

Schrijver, G., Sarmah, D. K., and El-Hajj, M. (2024). Automobile insurance fraud detection using data mining: A systematic literature review. *Intelligent Systems with Applications*, 21:200340.

Shannon, C. E. (1948). A Mathematical Theory of Communication. *Bell System Technical Journal*, 27(3):379–423.

Vorobyev, I. (2024). Fraud risk assessment in car insurance using claims graph features in machine learning. *Expert Systems with Applications*, 251:124109.

West, J. and Bhattacharya, M. (2016). Intelligent financial fraud detection: A comprehensive review. *Computers & Security*, 57:47–66.

Xu, D. and Tian, Y. (2015). A comprehensive survey of clustering algorithms. *Annals of Data Science*, 2:165–193.